

Step 1 — Install Kubeflow Pipelines SDK

Since you're using Windows, open:

Anaconda Prompt

I recommend creating a separate environment.

Create environment

```
conda create -n kubeflow python=3.11 -y
```

Activate:

```
conda activate kubeflow
```

Then install KFP:

```
pip install kfp
```

This follows the current Kubeflow Pipelines getting-started approach.

Check installation:

```
python -c "import kfp; print(kfp.__version__)"
```

You should get something like:

2.x.x

Next: Hands-on 2 — Multi-Step Kubeflow Pipeline

1. Environment

```
mkdir -p ~/kubeflow-mlops
```

```
cd ~/kubeflow-mlops
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

2. Install

```
python -m pip install kfp==2.17.0
```

```
python -m pip install pandas scikit-learn joblib
```

PART-A Hello World with SubprocessRunner

Create the Python file: `python subprocess_hello.py`

```
from kfp import dsl
from kfp import local

@dsl.component
def hello(name: str) -> str:
    message = f"Hello from KFP SubprocessRunner, {name}!"
    print(message)
    return message

@dsl.pipeline
def hello_pipeline(name: str = "MLOps Students"):
    hello_task = hello(name=name)

local.init(
    runner=local.SubprocessRunner(
        use_venv=False
    ),
    raise_on_error=False
)

hello_pipeline(name="MLOps Students")
```

Step 1 — Run this command

From your current directory:

```
find ~/kubeflow-mlops/local_outputs -type f | sort
```

Step 2 — Also run

```
find ~/kubeflow-mlops/local_outputs -type f \( -name "*.log" -
-o -name "*.txt" -o -name "executor_output.json" \) -print
```

Then, if you see an `executor_output.json`, run:

```
cat ~/kubeflow-  
mlops/local_outputs/*/hello/executor_output.json
```

Linux file-permission problem use this:

Step 1 — Check ownership of local_outputs

Run:

```
ls -ld ~/kubeflow-mlops/local_outputs
```

Then:

```
ls -la ~/kubeflow-mlops/local_outputs
```

Step 2 — Test write permission

Run:

```
touch ~/kubeflow-mlops/local_outputs/test.txt
```

Then:

```
ls -l ~/kubeflow-mlops/local_outputs/test.txt
```

Remove the test:

```
rm ~/kubeflow-mlops/local_outputs/test.txt
```

Step 3 — Clean the old outputs

Since these are only practical files, let's start fresh:

```
rm -rf ~/kubeflow-mlops/local_outputs/*
```

Check:

```
ls -la ~/kubeflow-mlops/local_outputs
```

Part B: Simple ML Pipeline

Step 1 — Install ML libraries

```
python -m pip install pandas scikit-learn joblib
```

Then verify:

```
python -c "import pandas, sklearn, joblib; print('ML libraries OK')"
```

Step 2.1 — Create the Python file

Create a file: nano simple_ml_pipeline.py

```
from kfp import dsl
from kfp import local

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["scikit-learn"]
)
def load_data():
    from sklearn.datasets import load_iris

    iris = load_iris()

    print("Dataset loaded successfully!")
    print(f"Number of samples: {len(iris.data)}")
    print(f"Number of features: {iris.data.shape[1]}")
    print(f"Number of classes: {len(set(iris.target))}")

@dsl.pipeline
def ml_pipeline():
    load_data()

local.init(
    runner=local.SubprocessRunner()
)

ml_pipeline()
Step 2.2 – Run it
python simple_ml_pipeline.py
```

output:

```
(.venv) gopi@gopi:~/kubeflow-mlops$ python simple_ml_pipeline.py
04:12:32.258 - INFO - Running pipeline: 'ml-pipeline'
-----
04:12:32.264 - INFO - Executing task 'load-data'
04:12:32.265 - INFO - Streamed logs:

    [KFP Executor 2026-09-21 04:12:55,002 INFO]: Looking for component `load-data`
    [KFP Executor 2026-09-21 04:12:55,002 INFO]: Loading KFP component "load-data"
    [KFP Executor 2026-09-21 04:12:55,003 INFO]: Got executor_input:
    {
      "inputs": {},
      "outputs": {
        "outputFile": "/home/gopi/kubeflow-mlops/local_outputs/ml-pipeline-load-data-output"
      }
    }
    Dataset loaded successfully!
    Number of samples: 150
    Number of features: 4
    Number of classes: 3
    [KFP Executor 2026-09-21 04:12:56,107 INFO]: Wrote executor output file

04:12:56.452 - INFO - Task 'load-data' finished with status SUCCESS
04:12:56.452 - INFO - Task 'load-data' has no outputs
-----
04:12:56.453 - INFO - Pipeline 'ml-pipeline' finished with status SUCCESS
```

Replace your current file

Open:

nano simple_ml_pipeline.py

```
from kfp import dsl
from kfp import local
from kfp.dsl import Output, Dataset
```

```
@dsl.component(
    base_image="python:3.11",
    packages_to_install=["scikit-learn", "pandas"]
)
def load_data(dataset: Output[Dataset]):

    from sklearn.datasets import load_iris
    import pandas as pd
```

```

iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target

df.to_csv(
    dataset.path,
    index=False
)

print("Dataset loaded successfully!")
print(f"Number of samples: {len(df)}")
print(f"Number of features: {len(iris.data[0])}")
print(f"Number of classes: {len(set(iris.target))}")

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["pandas", "scikit-learn"]
)
def preprocess_data(
    input_dataset: dsl.Input[Dataset],
    processed_dataset: Output[Dataset]
):

    import pandas as pd
    from sklearn.model_selection import train_test_split

    # Read dataset
    df = pd.read_csv(input_dataset.path)

    # Separate features and target
    X = df.drop("target", axis=1)
    y = df["target"]

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )

```

```

# Save training data
train_df = X_train.copy()
train_df["target"] = y_train

# Save testing data
test_df = X_test.copy()
test_df["target"] = y_test

train_df.to_csv(
    processed_dataset.path + "_train.csv",
    index=False
)

test_df.to_csv(
    processed_dataset.path + "_test.csv",
    index=False
)

print("Preprocessing completed!")
print(f"Training samples: {len(train_df)}")
print(f"Testing samples: {len(test_df)}")

@dsl.pipeline
def ml_pipeline():

    # Step 1: Load data
    data_task = load_data()

    # Step 2: Preprocess data
    preprocess_task = preprocess_data(
        input_dataset=data_task.outputs["dataset"]
    )

local.init(
    runner=local.SubprocessRunner()
)

ml_pipeline()

Then add this component after preprocess_data():
@dsl.component(

```

```

    base_image="python:3.11",
    packages_to_install=["pandas", "scikit-learn", "joblib"]
)
def train_model(
    processed_dataset: dsl.Input[Dataset],
    model: dsl.Output[dsl.Model]
):

    import pandas as pd
    import joblib
    from sklearn.linear_model import LogisticRegression

    # Read training data
    train_file = processed_dataset.path + "_train.csv"

    train_df = pd.read_csv(train_file)

    X_train = train_df.drop("target", axis=1)
    y_train = train_df["target"]

    print("Training Logistic Regression model...")

    classifier = LogisticRegression(
        max_iter=200
    )

    classifier.fit(
        X_train,
        y_train
    )

    # Save model
    joblib.dump(
        classifier,
        model.path
    )

    print("Model training completed!")
    print(f"Training samples: {len(train_df)}")

```

Then modify your pipeline:

```

@dsl.pipeline
def ml_pipeline():

    # Step 1
    data_task = load_data()

```

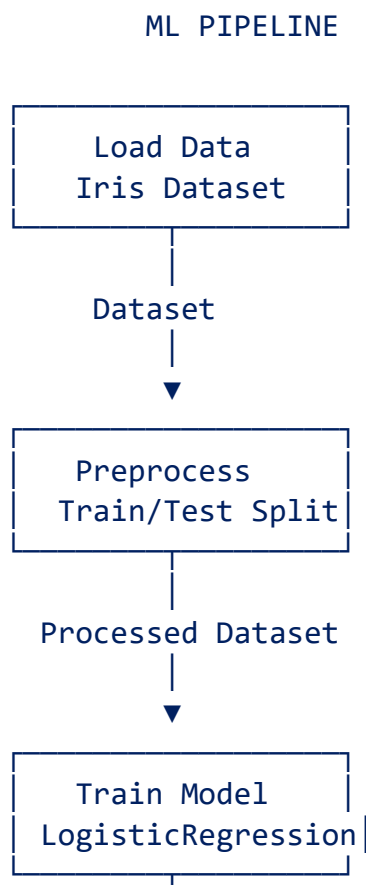
```
# Step 2
preprocess_task = preprocess_data(
    input_dataset=data_task.outputs["dataset"]
)
```

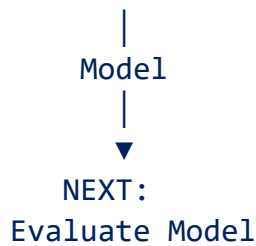
```
# Step 3
train_task = train_model(
    processed_dataset=preprocess_task.outputs[
        "processed_dataset"
    ]
)
```

```
Keep:
local.init(
    runner=local.SubprocessRunner()
)
```

```
ml_pipeline()
```

Your pipeline should now look like this





Three important KFP concepts:

1. Component

```
@dsl.component
def train_model(...):
```

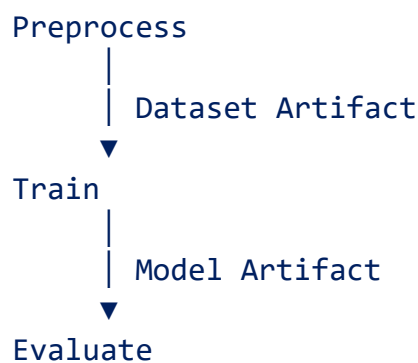
2. Artifact

```
model: dsl.Output[dsl.Model]
```

3. Dependency

```
processed_dataset=preprocess_task.outputs[
    "processed_dataset"
]
```

So KFP understands:



Add Evaluate Model

```
@dsl.component(
    base_image="python:3.11",
    packages_to_install=["pandas", "scikit-learn", "joblib"]
)
def evaluate_model(
    processed_dataset: dsl.Input[Dataset],
    model: dsl.Input[dsl.Model]
):

    import pandas as pd
    import joblib
    from sklearn.metrics import accuracy_score

    # Read test data
    test_file = processed_dataset.path + "_test.csv"
```

```

test_df = pd.read_csv(test_file)

X_test = test_df.drop("target", axis=1)
y_test = test_df["target"]

# Load trained model
classifier = joblib.load(model.path)

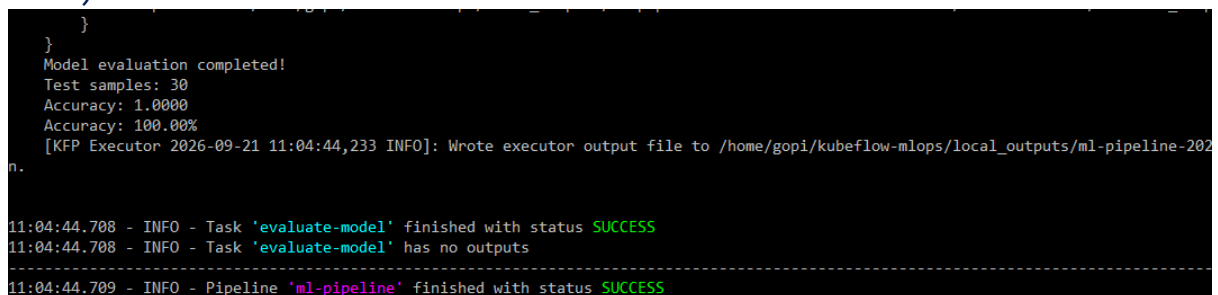
# Make predictions
predictions = classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(
    y_test,
    predictions
)

print("Model evaluation completed!")
print(f"Test samples: {len(test_df)}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Accuracy: {accuracy * 100:.2f}%")

# Step 4: Evaluate
evaluate_model(
    processed_dataset=preprocess_task.outputs[
        "processed_dataset"
    ],
    model=train_task.outputs["model"]
)

```



```

}
Model evaluation completed!
Test samples: 30
Accuracy: 1.0000
Accuracy: 100.00%
[KFP Executor 2026-09-21 11:04:44,233 INFO]: Wrote executor output file to /home/gopi/kubeflow-mlops/local_outputs/ml-pipeline-202
n.

11:04:44.708 - INFO - Task 'evaluate-model' finished with status SUCCESS
11:04:44.708 - INFO - Task 'evaluate-model' has no outputs
-----
11:04:44.709 - INFO - Pipeline 'ml-pipeline' finished with status SUCCESS

```

Compile the KFP ML Pipeline

run:

`nano compile_pipeline.py`

```

from kfp import compiler
from simple_ml_pipeline import ml_pipeline

```

```
compiler.Compiler().compile(  
    pipeline_func=ml_pipeline,  
    package_path="simple_ml_pipeline.yaml"  
)  
  
print("Pipeline compiled successfully!")  
print("Generated file: simple_ml_pipeline.yaml")
```

run:

```
python compile_pipeline.py
```

```
(.venv) gopi@gopi:~/kubeflow-mlops$ python compile_pipeline.py  
Pipeline compiled successfully!  
Generated file: simple_ml_pipeline.yaml
```

Check the YAML file

Run:

```
ls -lh simple_ml_pipeline.yaml
```

```
(.venv) gopi@gopi:~/kubeflow-mlops$ ls -lh simple_ml_pipeline.yaml  
-rw-r--r-- 1 gopi gopi 12K Sep 21 11:08 simple_ml_pipeline.yaml
```

You should see something like:

Then:

```
head -30 simple_ml_pipeline.yaml
```

```
(.venv) gopi@gopi:~/kubeflow-mlops$ head -30 simple_ml_pipeline.yaml
# PIPELINE DEFINITION
# Name: ml-pipeline
components:
  comp-evaluate-model:
    executorLabel: exec-evaluate-model
    inputDefinitions:
      artifacts:
        model:
          artifactType:
            schemaTitle: system.Model
            schemaVersion: 0.0.1
        processed_dataset:
          artifactType:
            schemaTitle: system.Dataset
            schemaVersion: 0.0.1
  comp-load-data:
    executorLabel: exec-load-data
    outputDefinitions:
      artifacts:
        dataset:
          artifactType:
            schemaTitle: system.Dataset
            schemaVersion: 0.0.1
  comp-preprocess-data:
    executorLabel: exec-preprocess-data
    inputDefinitions:
      artifacts:
        input_dataset:
          artifactType:
            schemaTitle: system.Dataset
```

Therefore, modify the bottom of simple_ml_pipeline.py

Change:

```
local.init(
    runner=local.SubprocessRunner()
)

ml_pipeline()
```

to:

```
if __name__ == "__main__":

    local.init(
        runner=local.SubprocessRunner()
    )
```

```
ml_pipeline()
```

This is an important Python concept:

`if __name__ == "__main__":` means execute this section only when the file is run directly, not when it is imported.

So:

Running directly: `python simple_ml_pipeline.py`

Practical 2 — Run the ML Pipeline with DockerRunner

Docker Desktop Installation Process — Windows + WSL2

Step 1: Check prerequisites

Before installing Docker Desktop, make sure:

- Windows 10/11 is installed
- **Virtualization is enabled** in BIOS
- **WSL2** is installed
- Ubuntu is installed in WSL2

Check WSL from **PowerShell**:

```
wsl --status
```

Then:

```
wsl -l -v
```

You should see something similar to:

NAME	STATE	VERSION
------	-------	---------

Ubuntu	Running	2
--------	---------	---

Step 2: Download Docker Desktop

Download Docker Desktop for Windows from the official Docker website:

[Docker Desktop for Windows \(https://www.docker.com/products/docker-desktop/?utm_source=chatgpt.com\)](https://www.docker.com/products/docker-desktop/?utm_source=chatgpt.com)

Choose **Docker Desktop for Windows**.

Step 3: Install Docker Desktop

Run the downloaded installer.

During installation, select:

☒ Use WSL 2 instead of Hyper-V

if that option is presented.

Then continue with the installation.

Restart Windows if Docker Desktop requests it.

Step 4 — Check Docker Desktop on Windows

Open Docker Desktop in Windows.

Go to:

Settings → General

Make sure:

☒ Use the WSL 2 based engine

is enabled.

Then click Apply & Restart if you changed it.

Step 5 — Enable Ubuntu integration

In Docker Desktop go to:

Settings → Resources → WSL Integration

You should see your distributions, including Ubuntu.

Enable:

- ☒ Enable integration with my default WSL distro
- ☒ Ubuntu

Then click:

Apply & Restart

Step 6 — Restart WSL

After Docker Desktop restarts, close your Ubuntu terminal.

Open **PowerShell** or **Command Prompt** in Windows and run:

```
wsl --shutdown
```

Then start Ubuntu again.

Step 7 — Check Ubuntu is WSL2

From PowerShell:

```
wsl -l -v
```

You should see something like:

```
NAME    STATE   VERSION
Ubuntu  Running 2
```

Docker requires the distribution to be running in WSL 2 mode for this integration.

Step 8 — Test Docker again

Back in Ubuntu:

```
docker --version
```

Then:

```
docker run hello-world
```

Expected:

```
Hello from Docker!
```

Step 9 — Test Python Docker SDK

Once the above works, return to your KFP environment:

```
cd ~/kubeflow-mlops
source .venv/bin/activate
```

Then:

```
python -c "import docker; print('Docker SDK:', docker.__version__)"
```

And finally:

```
python -c "import docker; c=docker.from_env(); print('Docker Engine:', c.version()['Version'])"
```

Next step — DockerRunner Hello World

create the file: nano docker_hello.py

```
from kfp import dsl
from kfp import local

@dsl.component(base_image="python:3.11")
def hello(name: str) -> str:
    message = f"Hello from KFP DockerRunner, {name}!"
    print(message)
    return message

@dsl.pipeline
def hello_pipeline(name: str = "MLOps Students"):
    hello_task = hello(name=name)

# Initialize KFP with DockerRunner
local.init(
    runner=local.DockerRunner()
)

# Run the pipeline
hello_pipeline(name="MLOps Students")
```

Then run:

python docker_hello.py

Next: — Run the ML Pipeline with DockerRunner

Create file: docker_ml_pipeline.py

```
from kfp import dsl
from kfp import local
from kfp.dsl import Output, Dataset

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["scikit-learn", "pandas"]
)
def load_data(dataset: Output[Dataset]):
```

```

from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target

df.to_csv(
    dataset.path,
    index=False
)

print("Dataset loaded successfully!")
print(f"Number of samples: {len(df)}")
print(f"Number of features: {len(iris.data[0])}")
print(f"Number of classes: {len(set(iris.target))}")

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["pandas", "scikit-learn"]
)
def preprocess_data(
    input_dataset: dsl.Input[Dataset],
    processed_dataset: Output[Dataset]
):

    import pandas as pd
    from sklearn.model_selection import train_test_split

    # Read dataset
    df = pd.read_csv(input_dataset.path)

    # Separate features and target
    X = df.drop("target", axis=1)
    y = df["target"]

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(
        X,

```

```

        y,
        test_size=0.2,
        random_state=42
    )

    # Save training data
    train_df = X_train.copy()
    train_df["target"] = y_train

    # Save testing data
    test_df = X_test.copy()
    test_df["target"] = y_test

    train_df.to_csv(
        processed_dataset.path + "_train.csv",
        index=False
    )

    test_df.to_csv(
        processed_dataset.path + "_test.csv",
        index=False
    )

    print("Preprocessing completed!")
    print(f"Training samples: {len(train_df)}")
    print(f"Testing samples: {len(test_df)}")

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["pandas", "scikit-learn", "joblib"]
)
def train_model(
    processed_dataset: dsl.Input[Dataset],
    model: dsl.Output[dsl.Model]
):

    import pandas as pd
    import joblib
    from sklearn.linear_model import LogisticRegression

    # Read training data
    train_file = processed_dataset.path + "_train.csv"

    train_df = pd.read_csv(train_file)

    X_train = train_df.drop("target", axis=1)

```

```

y_train = train_df["target"]

print("Training Logistic Regression model...")

classifier = LogisticRegression(
    max_iter=200
)

classifier.fit(
    X_train,
    y_train
)

# Save model
joblib.dump(
    classifier,
    model.path
)

print("Model training completed!")
print(f"Training samples: {len(train_df)}")

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["pandas", "scikit-learn", "joblib"]
)
def evaluate_model(
    processed_dataset: dsl.Input[Dataset],
    model: dsl.Input[dsl.Model]
):

    import pandas as pd
    import joblib
    from sklearn.metrics import accuracy_score

    # Read test data
    test_file = processed_dataset.path + "_test.csv"

    test_df = pd.read_csv(test_file)

    X_test = test_df.drop("target", axis=1)
    y_test = test_df["target"]

    # Load trained model
    classifier = joblib.load(model.path)

```

```

# Make predictions
predictions = classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(
    y_test,
    predictions
)

print("Model evaluation completed!")
print(f"Test samples: {len(test_df)}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Accuracy: {accuracy * 100:.2f}%")

@dsl.pipeline
def ml_pipeline():

    # Step 1: Load data
    data_task = load_data()

    # Step 2: Preprocess data
    preprocess_task = preprocess_data(
        input_dataset=data_task.outputs["dataset"]
    )

    # Step 3
    train_task = train_model(
        processed_dataset=preprocess_task.outputs[
            "processed_dataset"
        ]
    )

    # Step 4: Evaluate
    evaluate_model(
        processed_dataset=preprocess_task.outputs[
            "processed_dataset"
        ],
        model=train_task.outputs["model"]
    )

local.init(

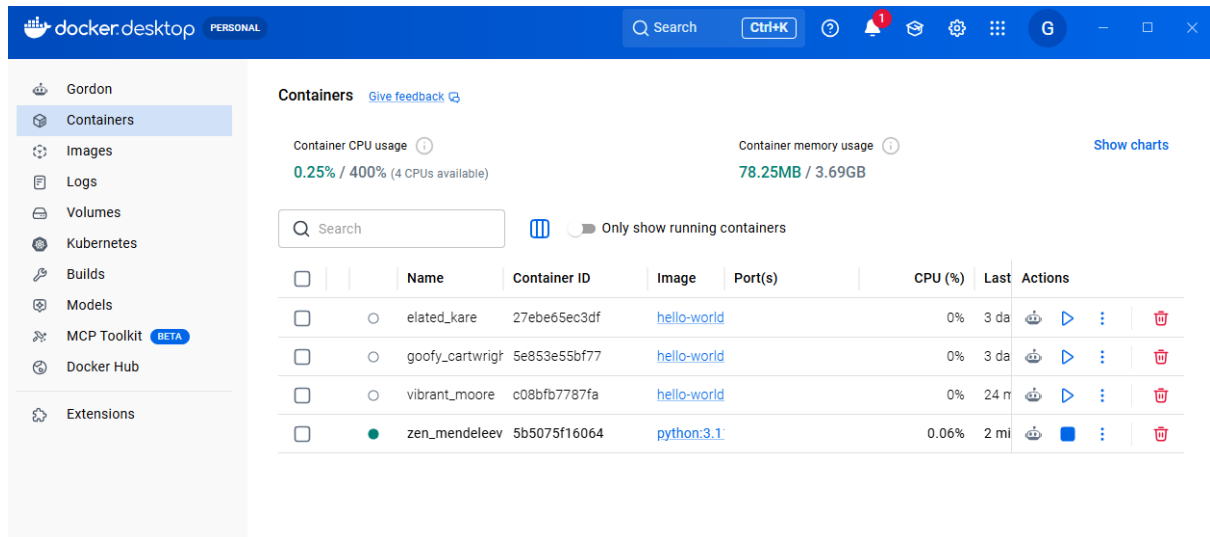
```

```
runner=local.DockerRunner()  
)
```

```
ml_pipeline()
```

Then run:

```
python docker_ml_pipeline.py
```



The screenshot shows the Docker Desktop application window. The left sidebar contains navigation options: Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Models, MCP Toolkit (marked BETA), Docker Hub, and Extensions. The main panel displays the 'Containers' section with a 'Give feedback' link. It shows overall usage: 'Container CPU usage 0.25% / 400% (4 CPUs available)' and 'Container memory usage 78.25MB / 3.69GB'. A 'Show charts' link is present. Below this is a search bar and a toggle for 'Only show running containers'. A table lists four containers:

☐	☐	Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
☐	☐	elated_kare	27ebe65ec3df	hello-world		0%	3 da	
☐	☐	goofy_cartwrigl	5e853e55bf77	hello-world		0%	3 da	
☐	☐	vibrant_moore	c08bf7787fa	hello-world		0%	24 m	
☐	☒	zen_mendeleev	5b5075f16064	python:3.1		0.06%	2 mi	